

Sección 3-1: Comparaciones Lógicas y Reglas de Prioridad

Objetivos:

- Evaluar las comparaciones lógicas para restringir las filas devueltas en función de dos o más condiciones.
- Aplicar las reglas de prioridad para determinar el orden en el que se evalúan y calculan las expresiones.

Objetivo:

- No hay muchas cosas en la vida que dependan de una sola condición.
- Por ejemplo, si desea ir a la universidad, probablemente necesite buenas notas y dinero para pagarla.
- Si tiene dinero adicional, puede guardarlo o gastarlo.
- Si desea ir al cine, es posible que no desee ir este fin de semana y puede que no desee sentarse en las primeras 10 filas de la sala.

Condiciones Lógicas:

- Las condiciones lógicas combinan el resultado de dos condiciones de componentes para producir un único resultado según dichas condiciones.
- Por ejemplo, para asistir a un concierto de rock, debe adquirir una entrada y el transporte para llegar allí.
- Si se cumplen las dos condiciones, puede ir al concierto. ¿Qué hacer si no puede obtener transporte, puede ir?

Operadores Lógicos:

- Un operador lógico combina los resultados de dos o más condiciones para producir un único resultado.
- Se devuelve un resultado SOLO SI el resultado global de la condición es verdadero.
 - AND: devuelve TRUE (verdadero) si ambas condiciones son verdaderas.
 - OR: devuelve TRUE (verdadero) si cualquier condición es verdadera.
 - NOT: devuelve TRUE (verdadero) si la condición es falsa.

Operador AND:

- En la siguiente consulta, los resultados devueltos serán las filas que cumplan ambas condiciones especificadas en la cláusula WHERE.

Unset

```
SELECT last_name, department_id, salary
FROM employees
WHERE department_id > 50 AND salary > 12000;
```

Operador AND:

- Otro ejemplo del uso de AND en la cláusula WHERE.

Unset

```
SELECT last_name, hire_date, job_id
FROM employees
WHERE hire_date > '01-Jan-1998' AND job_id LIKE 'SA%';
```

Operador OR:

- Si la cláusula WHERE utiliza la condición OR, los resultados devueltos de una consulta serán las filas que cumplan una de las condiciones de OR.
- En otras palabras, todas las filas devueltas tienen un location_id de 2500 o tienen un manager_id igual a 124.

Unset

```
SELECT department_name, manager_id, location_id
FROM departments
WHERE location_id = 2500 OR manager_id=124;
```

Operador NOT:

- El operador NOT devolverá las filas que no cumplan con la condición de la cláusula WHERE.

Unset

```
SELECT department_name, location_id
FROM departments
WHERE location_id NOT IN (1700, 1800);
```

Reglas de Prioridad o Qué Ocurre en Primer Lugar:

- Considere la siguiente sentencia SELECT. ¿En qué orden se evalúan las expresiones calculadas?
- Afortunadamente, cuando las cosas se complican de esta manera, SQL cuenta con algunas reglas básicas fáciles de seguir.

Unset

```
SELECT last_name || ' ' || salary * 1.05 AS "Employee Raise"
FROM employees
WHERE department_id IN (50, 80) AND first_name LIKE 'C%'
OR last_name LIKE '%s%';
```

Reglas de Prioridad o Qué Ocurre en Primer Lugar:

- Tenga en cuenta que el operador AND se evalúa antes que el operador OR.
- Esto significa que, para que el ejemplo en la diapositiva anterior, si no se cumple alguna de las condiciones de la sentencia AND, se utilizará el operador OR para seleccionar las filas.
- Es importante recordar este concepto.

Orden de las Operaciones:

1. Aritmética + - * /
2. Concatenación ||
3. Comparación <, <=, >, >=, <>
4. IS (NOT) NULL, LIKE, (NOT) IN
5. (NOT) BETWEEN
6. NOT
7. AND
8. OR

Reglas de Prioridad o Qué Ocurre en Primer Lugar:

- En primer lugar, se evalúa la condición AND, por lo que se devolverán todos los empleados que trabajen en el departamento 80 o 50, y tengan un nombre que empiece por "C".
- La cláusula OR se evalúa a continuación y se devuelven los empleados cuyo apellido contenga "s".

Unset

```
SELECT last_name || ' ' || salary * 1.05 AS "Employee Raise",  
department_id, first_name  
FROM employees  
WHERE department_id IN (50, 80) AND first_name LIKE 'C%'  
OR last_name LIKE '%s%';
```

Reglas de Prioridad o Qué Ocurre en Primer Lugar:

- En este ejemplo, el orden de los operadores OR y AND se ha invertido con respecto a la diapositiva anterior.

Unset

```
SELECT last_name || ' ' || salary * 1.05 AS "Employee Raise",  
department_id, first_name  
FROM employees  
WHERE department_id IN (50, 80) OR first_name LIKE 'C%'  
AND last_name LIKE '%s%';
```

Reglas de Prioridad o Qué Ocurre en Primer Lugar:

- Agregar paréntesis cambia la forma en la que se evalúa la cláusula WHERE y las filas devueltas.

Unset

```
SELECT last_name || ' ' || salary * 1.05 AS "Employee Raise",  
department_id, first_name  
FROM employees  
WHERE (department_id IN (50, 80) OR first_name LIKE 'C%')  
AND last_name LIKE '%s%';
```

Terminología:

- AND
- OR
- NOT
- Reglas de prioridad

Sección 3-2: Ordenación de Filas

Objetivos:

- Crear una consulta para ordenar un juego de resultados en orden ascendente o descendente.
- Establecer el orden en el que se evalúan y calculan las expresiones en función de las reglas de prioridad.
- Crear una consulta para ordenar un juego de resultados con un alias de columna.
- Crear una consulta para ordenar un juego de resultados para una o varias columnas.

Cláusula ORDER BY:

- La información ordenada en orden ascendente nos es conocida.
- Es lo que hace que consultar un número en una agenda, buscar una palabra en el diccionario, o localizar una casa por su dirección sea relativamente fácil.
- SQL utiliza la cláusula ORDER BY para ordenar datos.
- La cláusula ORDER BY puede especificar varias formas en las que ordenar las filas devueltas en una consulta.

Cláusula ORDER BY:

- El orden por defecto es el orden ascendente.
- Se muestran los valores numéricos del más bajo al más alto.

- Los valores de fecha se muestran con el valor más reciente en primer lugar.
- Los valores de caracteres se muestran en orden alfabético.
- Los valores nulos se muestran al final para las secuencias ascendentes y al principio para las secuencias descendentes.
- **NULLS FIRST** especifica que los valores nulos se deben devolver antes los valores no nulos.
- **NULLS LAST** especifica que los valores nulos se deben devolver después los valores no nulos.

Unset

```
SELECT last_name, hire_date, department_id
FROM employees
ORDER BY department_id NULLS LAST;
```

Cláusula ORDER BY:

- El siguiente ejemplo de empleados utiliza la cláusula ORDER BY para ordenar hire_date en orden ascendente (valor por defecto).
- Nota: La cláusula ORDER BY debe ser la última cláusula de la sentencia SQL.

Unset

```
SELECT last_name, hire_date
FROM employees
ORDER BY hire_date;
```

•

Unset

```
LAST_NAME    HIRE_DATE
```

King	17-Jun-1987
Whalen	17-Sep-1987
Kochhar	21-Sep-1989
Hunold	03-Jan-1990
Ernst	21-May-1991
De Haan	13-Jan-1993
Gietz	07-Jun-1994
Higgins	07-Jun-1994
Rajs	17-Oct-1995
Hartstein	17-Feb-1996

Ordenación en Orden Descendente:

- Puede invertir el orden por defecto en la cláusula ORDER BY para que aparezcan en orden descendente especificando la palabra clave DESC después del nombre de columna en la cláusula ORDER BY.

Unset

```
SELECT last_name, hire_date
FROM employees
ORDER BY hire_date DESC;
```

•

Unset

LAST_NAME	HIRE_DATE
Zlotkey	29-Jan-2000
Mourgos	16-Nov-1999
Grant	24-May-1999
Lorentz	07-Feb-1999
Vargas	09-Jul-1998
Taylor	24-Mar-1998

Matos	15-Mar-1998
Fay	17-Aug-1997
Davies	29-Jan-1997
Abel	11-May-1996

Uso de Alias de Columna:

- Puede ordenar los datos mediante un alias de columna.
- El alias utilizado en la sentencia SELECT es al que se hace referencia en la cláusula ORDER BY.

Unset

```
SELECT last_name, hire_date AS "Date Started"  
FROM employees  
ORDER BY "Date Started";
```

-

Unset

LAST_NAME	Date Started
King	17-Jun-1987
Whalen	17-Sep-1987
Kochhar	21-Sep-1989
Hunold	03-Jan-1990
Ernst	21-May-1991
De Haan	13-Jan-1993
Gietz	07-Jun-1994
Higgins	07-Jun-1994
Rajs	17-Oct-1995
Hartstein	17-Feb-1996

Ordenación con Otras Columnas:

- También es posible utilizar la cláusula ORDER BY para ordenar la salida según una columna que no aparezca en la cláusula SELECT.
- En el siguiente ejemplo, los datos se ordenan según la columna last_name, incluso aunque esta columna no aparezca en la sentencia SELECT.

Unset

```
SELECT employee_id, first_name  
FROM employees  
WHERE employee_id < 105  
ORDER BY last_name;
```

•

Unset

EMPLOYEE_ID	FIRST_NAME
102	Lex
104	Bruce
103	Alexander
100	Steven
101	Neena

Orden de Ejecución:

- El orden de ejecución de una sentencia SELECT es el siguiente:
 - Cláusula FROM: busca la tabla que contiene los datos.
 - Cláusula WHERE: restringe las filas que se van a devolver.
 - Cláusula SELECT: selecciona en el conjunto de datos reducido las columnas solicitadas.
 - Cláusula ORDER BY: ordena el conjunto de resultados.

Ordenación con Varias Columnas:

- También se puede ordenar los resultados de la consulta por más de una columna.

- De hecho, no hay ningún límite en el número de columnas que se pueden agregar a la cláusula ORDER BY.

Ordenación con Varias Columnas:

- A continuación se muestra un ejemplo de ordenación con varias columnas.
- Los empleados se ordenan primero por número de departamento (del más bajo al más alto), a continuación, se muestran los apellidos en orden alfabético (A-Z).

Unset

```
SELECT department_id, last_name
FROM employees
WHERE department_id <= 50
ORDER BY department_id, last_name;
```

•

Unset

DEPARTMENT_ID	LAST_NAME
10	Whalen
20	Fay
20	Hartstein
50	Davies
50	Matos
50	Mourgos
50	Rajs
50	Vargas

Ordenación con Varias Columnas:

- Para crear una cláusula ORDER BY a fin de ordenar por varias columnas, especifique las columnas que se van a devolver y separe los nombres de columna mediante comas.

- Si desea invertir el orden de ordenación de una columna, agregue DESC después del nombre.

Unset

```
SELECT department_id, last_name  
FROM employees  
WHERE department_id <= 50  
ORDER BY department_id DESC, last_name;
```

•

Unset

DEPARTMENT_ID	LAST_NAME
50	Davies
50	Matos
50	Mourgos
50	Rajs
50	Vargas
20	Fay
20	Hartstein
10	Whalen

Terminología:

- Cláusula ORDER BY
- ASCENDENTE
- DESCENDENTE
- Orden de Ejecución

Sección 3-3: Introducción a las Funciones

Objetivos:

- Identificar las aplicaciones adecuadas de funciones de fila única en las sentencias de consulta.

- Clasificar una función como función de una sola fila o de varias filas.
- Diferenciar entre funciones de una sola fila y funciones de varias filas y los resultados devueltos por cada una.

Objetivo:

- Al introducir dinero en una máquina de bebidas, ocurre algo entre el momento en el que se deposita el dinero y en el que se dispensa su bebida favorita.
- La máquina procesa la transacción internamente.
- Su dinero es la entrada y la bebida es la salida.
- La máquina realiza una función:
 - Cuenta su dinero.
 - Se asegura de que ha seleccionado su elección.
 - Devuelve cambio, si es necesario.

Objetivo:

- En SQL, existen muchos tipos de funciones que se utilizan para transformar la entrada de una forma en la salida de otra forma.
- Estas funciones se utilizan para manipular los valores de datos.
- Las funciones son pequeños programas que realizan una acción en un valor o columna y producen algo distinto como salida.

Funciones:

- Las funciones tienen tanto entrada como salida. La entrada en una función se conoce como argumento.
- En el ejemplo de la máquina de bebidas, la entrada es el dinero y la salida es una bebida.

Funciones:

- Oracle cuenta con dos tipos de funciones diferentes:
 - De una sola fila.
 - De varias filas.
- *Funciones de una Sola Fila frente a

Funciones de Varias Filas:**

- Las funciones de una sola fila funcionan solo en filas únicas y devuelven un resultado por fila.
- Existen distintos tipos de funciones de una sola fila, como funciones de carácter, número, fecha y conversión.
- Las funciones de varias filas pueden manipular grupos de filas para proporcionar un resultado por grupo de filas.
- Estas funciones también se conocen como funciones de grupo.

Funciones de una Sola Fila:

- En SQL, las funciones de una sola fila se pueden utilizar para:
 - Realizar cálculos como redondear números para una determinada posición decimal.
 - Modificar elementos de datos individuales como la conversión de valores de caracteres de mayúsculas a minúsculas.
 - Dar formato a fechas y números para su visualización como, por ejemplo, convertir el formato de fecha de una base de datos numérico interno en un formato estándar.
 - Convertir los tipos de dato de columna, como la conversión de una cadena de caracteres en un número o fecha.

Funciones de una Sola Fila:

- Las funciones de una sola fila aceptan uno o más argumentos y devolverán un único resultado por fila.
- Por lo tanto, si se aplica la función de una sola fila a 12 filas, obtendrá 12 resultados de la función de una sola fila.
- En resumen, las funciones de una sola fila hacen lo siguiente:
 - Manipular elementos de datos.
 - Aceptar argumentos y devolver un valor.
 - Actuar en cada fila devuelta.
 - Devolver un resultado por fila.
 - Posibilidad de modificar el tipo de dato.
 - Posibilidad de anidamiento.

Funciones de Varias Filas:

- Las funciones de varias filas (o de grupo) toman muchas filas como entrada, y devuelven un valor único como salida.
- La entrada de filas puede ser toda la tabla o la tabla dividida en grupos más pequeños.
- Los ejemplos de las funciones de varias filas (de grupo) incluyen:
 - **MAX**: busca el valor más alto en un grupo de filas.
 - **MIN**: busca el valor más bajo en un grupo de filas.
 - **AVG**: busca el valor medio de un grupo de filas.

Terminología:

- Función de una sola fila.
- Función de varias filas.

Resumen:

- Identificar las aplicaciones adecuadas de funciones de fila única en las sentencias de consulta.

- Clasificar una función como función de una sola fila o de varias filas.
 - Diferenciar entre funciones de una sola fila y funciones de varias filas y los resultados devueltos por cada una.
-

Sección 4 -1: Manipulación de Mayúsculas/Minúsculas y de Caracteres

Objetivos

- Seleccionar y aplicar funciones de una sola fila que realicen conversión de mayúsculas/minúsculas y/o la manipulación de caracteres.
 - Seleccionar y aplicar las funciones de manipulación de mayúsculas/minúsculas de caracteres **LOWER**, **UPPER** e **INITCAP** en una consulta SQL.
 - Seleccionar y aplicar las funciones de manipulación de caracteres **CONCAT**, **SUBSTR**, **LENGTH**, **INSTR**, **LPAD**, **RPAD**, **TRIM** y **REPLACE** en una consulta SQL.
 - Escribir consultas flexibles usando variables de sustitución.
-

Objetivo

- Cambiar la forma en la que se presentan los datos es importante cuando se trabaja con datos de una base de datos.
 - En SQL, es frecuente cambiar la forma en la que aparecen los datos según los requisitos de la tarea.
 - Aprender diversas formas de transformar los datos para que se ajusten a una situación concreta.
-

Tabla DUAL

- La tabla DUAL tiene una fila denominada "X" y una columna denominada "DUMMY".
- Se utiliza para crear sentencias **SELECT** y ejecutar funciones que no están directamente relacionadas con una tabla de base de datos concreta.
- Las consultas que utilizan la tabla DUAL devuelven una fila como resultado.

Ejemplo

Unset

```
SELECT (319/29) + 12 FROM DUAL;
```

Funciones de Caracteres de Una Sola Fila

- Las funciones de caracteres de una sola fila se dividen en dos categorías:
 - Funciones que convierten las mayúsculas/minúsculas de las cadenas de caracteres.
 - Funciones que permiten unir, extraer, mostrar, encontrar, rellenar y recortar cadenas de caracteres.
- Las funciones de una sola fila se pueden utilizar en las cláusulas **SELECT**, **WHERE** y **ORDER BY**.

Funciones de Manipulación de Mayúsculas/Minúsculas

Unset

```
graph TD;
    A[Funciones de Manipulación de Mayúsculas/Minúsculas] --> B[LOWER];
    A --> C[UPPER];
    A --> D[INITCAP];
```

- Permiten convertir temporalmente los datos en mayúsculas y minúsculas.
- Ejemplo:

Unset

```
SELECT last_name FROM employees WHERE LOWER(last_name) = 'abel';
SELECT last_name FROM employees WHERE UPPER(last_name) = 'ABEL';
SELECT last_name FROM employees WHERE INITCAP(last_name) =
'Abe1';
```

Funciones de Manipulación de Caracteres

Unset

```
graph TD;
  A[Funciones de Manipulación de Caracteres] --> B[CONCAT];
  A --> C[SUBSTR];
  A --> D[LENGTH];
  A --> E[INSTR];
  A --> F[LPAD];
  A --> G[RPAD];
  A --> H[TRIM];
  A --> I[REPLACE];
```

CONCAT

- Une dos valores.
- Ejemplo:

Unset

```
SELECT CONCAT('Hello', 'World') FROM DUAL;
```

SUBSTR

- Extrae una cadena de una longitud determinada.
- Ejemplo:

Unset

```
SELECT SUBSTR('HelloWorld', 1, 5) FROM DUAL;
```

LENGTH

- Muestra la longitud de una cadena como un valor numérico.

- Ejemplo:

Unset

```
SELECT LENGTH('HelloWorld') FROM DUAL;
```

INSTR

- Encuentra la posición numérica de los caracteres especificados.
- Ejemplo:

Unset

```
SELECT INSTR('HelloWorld', 'W') FROM DUAL;
```

LPAD

- Rellena la parte izquierda de una cadena de caracteres.
- Ejemplo:

Unset

```
SELECT LPAD('HelloWorld', 15, '-') FROM DUAL;
```

RPAD

- Rellena la parte derecha de una cadena de caracteres.
- Ejemplo:

Unset

```
SELECT RPAD('HelloWorld', 15, '-') FROM DUAL;
```

TRIM

- Elimina todos los caracteres especificados.

- Ejemplo:

Unset

```
SELECT TRIM(LEADING 'a' FROM 'abcba') FROM DUAL;
```

REPLACE

- Sustituye una secuencia de caracteres por otra.
- Ejemplo:

Unset

```
SELECT REPLACE('JACK and JUE', 'J', 'BL') FROM DUAL;
```

Uso de Alias de Columna con Funciones

- Un alias de columna se utiliza a menudo para asignar un nombre a una función.
- Ejemplo:

Unset

```
SELECT LOWER(last_name) || LOWER(SUBSTR(first_name,1,1)) AS "User  
Name" FROM employees;
```

Variables de Sustitución

- Permiten ejecutar la misma consulta con muchos valores diferentes.
- Uso en Oracle Application Express mediante `:named_variable`.
- Ejemplo:

Unset

```
SELECT first_name, last_name, salary, department_id FROM  
employees WHERE department_id = :enter_dept_id;
```

Terminología

- **Funciones de carácter:** CONCAT, DUAL, Expresión, Formato, INITCAP, INSTR, LENGTH, LOWER, LPAD, REPLACE, RPAD, SUBSTR, TRIM, UPPER, Variable de sustitución.
-

Resumen

- Aplicación de funciones de una sola fila para manipular mayúsculas/minúsculas y caracteres.
 - Uso de variables de sustitución para consultas flexibles.
-

Sección 4 -2: Funciones Numéricas en SQL

1. Objetivos

- **Funciones a aplicar:** ROUND, TRUNC, MOD.
 - **Diferenciación de funciones:** Comparar los resultados de TRUNC y ROUND.
 - **Impacto en el negocio:** Entender las consecuencias de aplicar TRUNC y ROUND a valores numéricos.
-

2. Objetivo

- **Razón para el interés bancario:** Los bancos ajustan el tipo de interés basado en indicadores económicos.

- **Expresión del interés:** Los tipos de interés generalmente se expresan en porcentajes, como 3.45%.
-

3. Objetivo

- **Efecto del redondeo:** Considerar si redondear la tasa de interés a 3.5% sería beneficioso.
 - **Efecto del truncamiento:** Considerar si truncar a 3% sería aceptable.
 - **Importancia en bases de datos:** El truncado y redondeo de números es esencial en bases de datos para decisiones de negocio.
-

4. Funciones Numéricas

- **Funciones principales:**
 - ROUND
 - TRUNC
 - MOD
-

5. ROUND

- **Uso:** Redondear números y fechas.
- **Principal aplicación:** Redondeo a un número especificado de posiciones decimales.
- **Sintaxis:**

Unset

```
ROUND(column|expression, decimal places)
```

- **Ejemplos:**

Unset

```
ROUND(45.926) -- Resulta en 46  
ROUND(45.926, 0) -- Resulta en 46
```

6. ROUND

- **Redondeo a la derecha:** Posiciones decimales positivas redondean a la derecha de la coma decimal.

Unset

```
ROUND(45.926, 2) -- Resulta en 45.93
```

- **Redondeo a la izquierda:** Posiciones decimales negativas redondean a la izquierda de la coma decimal.

Unset

```
ROUND(45.926, -1) -- Resulta en 50
```

7. TRUNC

- **Uso:** Terminar números y fechas en posiciones decimales especificadas.
- **Comportamiento por defecto:** Sin posiciones decimales especificadas, se trunca en cero.
- **Sintaxis:**

Unset

```
TRUNC(column|expression, decimal places)
```

-

Ejemplo:

Unset

```
TRUNC(45.926, 2) -- Resulta en 45.92
```

8. TRUNC

- **Truncamiento por defecto:** Sin especificar decimales, trunca en cero.

Unset

```
TRUNC(45.926, 0) -- Resulta en 45
```

```
TRUNC(45.926) -- Resulta en 45
```

-

Nota importante: `TRUNC` no redondea; simplemente trunca.

9. MOD

- **Función:** Encuentra el resto de la división de un valor por otro.
- **Detección de paridad:** Útil para determinar si un número es par o impar.
 - Ejemplo: `MOD` de 5 dividido entre 2 es 1, indicando impar.

10. MOD

- **Demostración:** Muestra la paridad del número de aeropuertos en cada país.
- **Interpretación:**
 - 1 indica impar.
 - 0 indica par.
- **Ejemplo SQL:**

Unset

```
SELECT country_name, MOD(airports, 2) AS "Mod Demo"  
FROM wf_countries;
```

COUNTRY_NAME	Mod Demo
Canada	1
Republic of Costa Rica	0
Republic of Cape Verde	1
Greenland	0
Dominican Republic	0
State of Eritrea	1

11. Terminología

- **Términos clave:**
 - Funciones de número
 - MOD
 - ROUND
 - TRUNC

12. Resumen

- **Aprendizaje clave:**
 - Aplicación de **ROUND**, **TRUNC**, y **MOD** en SQL.
 - Diferenciación de **TRUNC** y **ROUND**.
 - Impacto en el negocio de estas funciones.
-

Sección 4 -3: Funciones de Fecha

Objetivos

- Mostrar el uso de **SYSDATE** y las funciones de fecha.
 - Establecer las consecuencias de negocio mundial para poder manipular fácilmente los datos almacenados en formato de fecha.
-

Objetivo

- ¿Se ha preguntado alguna vez cuántos días quedan del año escolar o el número de semanas que falta hasta la graduación?
 - Debido a que la base de datos de Oracle almacena las fechas como números, puede realizar cálculos en las fechas con los operadores matemáticos de suma y resta, entre otros.
 - Las empresas dependen de poder utilizar funciones de fecha para planificar las nóminas y los pagos, realizar un seguimiento de las revisiones de rendimiento de los empleados y años de servicio, o realizar un seguimiento de los pedidos y envíos.
 - Todas estas necesidades de negocio se realizan fácilmente mediante sencillas funciones de fecha de SQL.
-

Visualización de Fechas

- La visualización por defecto y el formato de entrada de las fechas es DD-Mon-YYYY.
- Por ejemplo: 02-Dec-2014.
- Sin embargo, las bases de datos Oracle almacenan fechas de modo interno en un formato numérico que representa el siglo, el año, el mes, el día, la hora, el minuto y el segundo.
- Las fechas de Oracle válidas son del 1 de enero de 4712 A.C. al 31 de diciembre de 9999 D.C.

- Esto representa el rango de fechas que puede almacenar correctamente en una base de datos Oracle.
-

SYSDATE

- **SYSDATE** es una función de fecha que devuelve la fecha y hora actuales del servidor de base de datos.
- Utilice **SYSDATE** para mostrar la fecha actual, utilizando la tabla **DUAL**.

Unset

```
SELECT SYSDATE FROM dual;
```

- **Ejemplo de resultado:**
 - **SYSDATE:** 01-Jul-2017
-

Tipo de Dato DATE

- El tipo de dato **DATE** siempre almacena internamente la información del año como un número de cuatro dígitos: dos dígitos para el siglo y dos dígitos para el año.
 - Por ejemplo, Oracle Database almacena el año como 1996 o 2004, no solo como 96 o 04.
 - En las versiones anteriores, el componente de siglo no se mostraba por defecto.
 - Sin embargo, debido a las cambiantes necesidades de los negocios en todo el mundo, el año de 4 dígitos es ahora la visualización por defecto.
-

Trabajar con Fechas

Ejemplos:

1. Agregar 60 días a **hire_date**:

Unset

```
SELECT last_name, hire_date + 60  
FROM employees;
```

2.

Mostrar el número de semanas desde que se contrató al empleado:

Unset

```
SELECT last_name, (SYSDATE - hire_date)/7  
FROM employees;
```

3.

Buscar el número de días que un empleado mantuvo un trabajo y mostrarlo en años:

Unset

```
SELECT employee_id, (end_date - start_date)/365 AS "Tenure in  
last job"  
FROM job_history;
```

Funciones de Fecha

Funciones Principales:

Función	Descripción
<code>MONTHS_BETWEEN</code>	Número de meses entre dos fechas
<code>ADD_MONTHS</code>	Agregar meses de calendario a fecha

NEXT_DAY	Fecha de la siguiente incidencia del día de la semana especificado
LAST_DAY	Último día del mes
ROUND	Redondear fecha
TRUNC	Truncar fecha

Nota: Todas las funciones de fecha devuelven un valor del tipo de dato **DATE** excepto la función **MONTHS_BETWEEN**, que devuelve un valor de tipo de dato numérico.

Funciones de Fecha: **MONTHS_BETWEEN**

- **Descripción:** Toma 2 argumentos **DATE** y devuelve el número de meses de calendario entre las 2 fechas.
- Si el primer argumento es una fecha anterior a la segunda, el número devuelto es negativo.

Ejemplo:

```
Unset
SELECT last_name, hire_date
FROM employees
WHERE MONTHS_BETWEEN (SYSDATE, hire_date) > 240;
```

Resultado:

last_name	hire_date
King	17-Jun-1987
Kochhar	21-Sep-1989
De Haan	13-Jan-1993

...

...

Funciones de Fecha: **ADD_MONTHS**

- **Descripción:** Toma 2 argumentos, una fecha y un número. Devuelve un valor de fecha con el argumento numérico agregado al componente mensual de la fecha.
- Si el número proporcionado es negativo, la función restará ese número de meses del argumento de fecha.

Ejemplo:

Unset

```
SELECT ADD_MONTHS(SYSDATE, 12) AS "Next Year"  
FROM dual;
```

Resultado: 01-Jul-2016

Funciones de Fecha: **NEXT_DAY**

- **Descripción:** Toma 2 argumentos, una fecha y un día de la semana y devuelve la fecha de la siguiente incidencia de ese día de la semana después del argumento **DATE**.

Ejemplo:

Unset

```
SELECT NEXT_DAY(SYSDATE, 'Saturday') AS "Next Saturday"  
FROM dual;
```

Resultado: 08-Jul-2017

Funciones de Fecha: **LAST_DAY**

- **Descripción:** Toma un argumento **DATE** y devuelve la fecha del último día del mes del argumento **DATE**.

Ejemplo:

```
Unset
SELECT LAST_DAY(SYSDATE) AS "End of the Month"
FROM dual;
```

Resultado: 31-Jul-2015

Funciones de Fecha: **ROUND**

- **Descripción:** Devuelve una fecha redondeada a la unidad especificada en el segundo argumento.

Ejemplos:

1. **Redondear al mes:**

```
Unset
SELECT hire_date, ROUND(hire_date, 'Month')
FROM employees
WHERE department_id = 50;
```

2.

Resultado:

hire_date	ROUND(hire_date, 'Month')
16-Nov-1999	01-Dec-1999
17-Oct-1995	01-Nov-1995

29-Jan-1997	01-Feb-1997
-------------	-------------

...	...
-----	-----

3.

Redondear al año:

Unset

```
SELECT hire_date, ROUND(hire_date, 'Year')
FROM employees
WHERE department_id = 50;
```

4.

Resultado:

hire_date	ROUND(hire_date, 'Year')
16-Nov-1999	01-Jan-2000
17-Oct-1995	01-Jan-1996
29-Jan-1997	01-Jan-1997
...	...

Funciones de Fecha: **TRUNC**

- **Descripción:** Devuelve una fecha truncada a la unidad especificada en el segundo argumento.

Ejemplos:

1. **Truncar al mes:**

Unset

```
SELECT hire_date, TRUNC(hire_date, 'Month')
FROM employees;
```

2.

Truncar al año:

Unset

```
SELECT hire_date, TRUNC(hire_date, 'Year')  
FROM employees;
```

Nota:

- Si 'month' es la unidad especificada, la fecha devuelta será el primer día del mes del argumento `DATE`.
- Si 'Year' es la unidad especificada, `TRUNC` devolverá el primer día del año del argumento `DATE`.

Ejemplo Completo de Funciones de Fecha

Consulta SQL:

Unset

```
SELECT employee_id, hire_date,  
ROUND(MONTHS_BETWEEN(SYSDATE, hire_date)) AS TENURE,  
ADD_MONTHS(hire_date, 6) AS REVIEW,  
NEXT_DAY(hire_date, 'FRIDAY'),  
LAST_DAY(hire_date)  
FROM employees  
WHERE MONTHS_BETWEEN(SYSDATE, hire_date) > 36;
```

Juego de Resultados:

Juego de Resultados:

EMPLOYEE_ID	HIRE_DATE	TENURE	REVIEW	NEXT_DAY(HIRE_DATE,'FRIDAY')	LAST_DAY(HIRE_DATE)
100	17-Jun-1987	348	17-Dec-1987	19-Jun-1987	30-Jun-1987
101	21-Sep-1989	321	21-Mar-1990	22-Sep-1989	30-Sep-1989
102	13-Jan-1993	281	13-Jul-1993	15-Jan-1993	31-Jan-1993
200	17-Sep-1987	345	17-Mar-1988	18-Sep-1987	30-Sep-1987
205	07-Jun-1994	265	07-Dec-1994	10-Jun-1994	30-Jun-1994
206	07-Jun-1994	265	07-Dec-1994	10-Jun-1994	30-Jun-1994
149	29-Jan-2000	197	29-Jul-2000	04-Feb-2000	31-Jan-2000
174	11-May-1996	241	11-Nov-1996	17-May-1996	31-May-1996
176	24-Mar-1998	219	24-Sep-1998	27-Mar-1998	31-Mar-1998
178	24-May-1999	205	24-Nov-1999	28-May-1999	31-May-1999

Terminología

- **Términos Clave:**
 - ADD_MONTHS
 - LAST_DAY
 - MONTHS_BETWEEN
 - NEXT_DAY
 - SYSDATE
 - ROUND
 - TRUNC

Resumen

- **Lo aprendido:**
 - Seleccionar y aplicar las funciones de una sola fila `MONTHS_BETWEEN`, `ADD_MONTHS`, `NEXT_DAY`, `LAST_DAY`, `ROUND` y `TRUNC` que funcionan en los datos de fecha.
 - Explicar cómo transforman las funciones de fecha las fechas de Oracle en datos de fecha o en valores numéricos.
 - Mostrar un uso adecuado de los operadores aritméticos con fechas.
 - **Funciones de fecha adicionales:**
 - Mostrar el uso de `SYSDATE` y las funciones de fecha.
 - Establecer las consecuencias de negocio mundial para manipular fácilmente los datos almacenados en formato de fecha.
-